



DD2395 Computer Security

DD2391 Cybersecurity Overview

Exam 2025-12-18 14:00–18:00

The problems are *not* sorted according to any perceived difficulty.

Students are allowed to bring at most 10 A4 papers (20 pages in total with student's full name written on every page) of handwritten or printed material to the exam. No electronic devices of any kind are allowed.

The exam grade is computed as follows:

- FX: 32-34 points
- E: 35-44 points
- D: 45-54 points
- C: 55-64 points
- B: 65-74 points
- A: 75-100 points

1 Symmetric Cryptography - Mode of Encryption [12 points]

Alice and Bob design the AES-MIX mode of operation as follows:

- $C_1 = E_K(IV_1 \oplus P_1)$
- $C_2 = E_K(IV_2 \oplus P_2) \oplus C_1$
- $C_i = E_K(C_{i-2} \oplus P_i)$, for $i > 2$

Here, $E_K(b)$ denotes the encryption of a block b of size n with secret key K (shared between Alice and Bob) using a secure block cipher E , for example AES. When Alice sends a message Bob, the proposed mode AES-MIX will split a message P into m blocks $(P_1, \dots, P_m)$ of size n , choose two independent random initialization vectors IV_1 and IV_2 , and send $(IV_1, IV_2, C_1, \dots, C_m)$ to Bob. You can assume that the message P is a multiple of n , hence no padding is needed.

- 1a [4 points]** Help Bob to derive an efficient decryption algorithm/formula for the block P_2 and for the block P_i , for $i > 2$. If needed, you can use $D_K(b)$ to denote the decryption of a block b of size n with encryption key K .

Solution The decryption formula for P_2 is $P_2 = D_K(C_2 \oplus C_1) \oplus IV_2$. The formula can be derived by XORing or decrypting both sides, as follows:

$$\begin{aligned} C_2 &= E_K(IV_2 \oplus P_2) \oplus C_1 \\ C_2 \oplus C_1 &= E_K(IV_2 \oplus P_2) \\ D_K(C_2 \oplus C_1) &= IV_2 \oplus P_2 \\ D_K(C_2 \oplus C_1) \oplus IV_2 &= P_2 \end{aligned}$$

The decryption formula for P_i is $P_i = D_K(C_i) \oplus C_{i-2}$. The formula can be derived by XORing or decrypting both sides, as follows:

$$\begin{aligned} C_i &= E_K(C_{i-2} \oplus P_i) \\ D_K(C_i) &= C_{i-2} \oplus P_i \\ D_K(C_i) \oplus C_{i-2} &= P_i \end{aligned}$$

1b [2 points] Explain whether or not the encryption formula and the decryption formula can be parallelized.

Solution The encryption formula cannot be parallelized because the computation of C_i requires knowing C_{i-2} .

The decryption formula can be parallelized because the computation of P_i does not require knowing any of $P_1, \dots, P_{i-1}$.

1c [6 points] Alice has a three block message $P = (P_1, P_2, P_3)$, which she encrypts with AES-MIX to obtain the ciphertext $C = (IV_1, IV_2, C_1, C_2, C_3)$ and sends it to Bob. Unfortunately, a network error in transit causes a single bit flip on the first ciphertext block such that $C'_1 = C_1 \oplus 1$, where $C_1 \oplus 1$ denotes the single bit flip in C_1 .

Your task is to show what message will Bob compute when he decrypts the modified ciphertext $C' = (IV_1, IV_2, C'_1, C_2, C_3)$. If the message or part of the message are unpredictable (garbage) output, you should justify why, otherwise you should compute the actual (part of the) message in relation to the bit flip.

Solution When Bob decrypts the ciphertext $C' = (IV, C_1 \oplus 1, C_2, C_3)$, the following happens:

$$\begin{aligned} P'_1 &= D_K(C'_1) \oplus IV_1 \\ P'_1 &= D_K(C_1 \oplus 1) \oplus IV_1 \\ P'_1 &= \textit{Garbage} \oplus IV_1 \\ P'_1 &= \textit{Garbage} \end{aligned}$$

This implies that Bob will only see an unpredictable output in the first block.

For P'_2 , we have:

$$\begin{aligned} P'_2 &= D_K(C_2 \oplus C'_1) \oplus IV_2 \\ P'_2 &= D_K(C_2 \oplus C_1 \oplus 1) \oplus IV_2 \\ P'_2 &= \textit{Garbage} \oplus IV_2 \\ P'_2 &= \textit{Garbage} \end{aligned}$$

This implies that Bob will only see an unpredictable output in the second block.

For P'_3 , we have:

$$\begin{aligned}P'_3 &= D_K(C_3) \oplus C'_1 \\P'_3 &= D_K(C_3) \oplus C_1 \oplus 1 \\P'_3 &= P_3 \oplus 1\end{aligned}$$

This implies that Bob will see an output that is similar to the original P_3 with only one bit flipped.

2 Access Control - BLP Model [12 points]

You are a security consultant tasked with reviewing the access control system of an organization that uses the Bell-LaPadula (BLP) security model. Users and objects are categorized based on sensitivity (security clearance) levels and compartments. Recall that in the BLP model, labels L are pairs (SL, C) of sensitivity levels SL and compartments (sets of areas) C , and they are used to enforce the "no read up" and "no write down" access rules. The system you are reviewing has the following components:

- Three security clearance levels: Top Secret (TS), Secret (S), Confidential (C), and Public (P), such that $P < C < TS$ and $P < S < TS$.
- Two compartments: Academia (A) and Industry (I).

2a [6 points] Considering the system above, provide the definition of "no read up" and "no write down" rules for a user U with label $L_U = (SL_U, C_U)$ and an object O with label $L_O = (SL_O, C_O)$ (2 points). Draw the label partial order that meets the requirements of the organization (4 points).

Solution No read up: U may read O iff $L_O \sqsubseteq L_U$. No write down: U may write O iff $L_U \sqsubseteq L_O$. For two labels $L_1 = (SL_1, C_1)$ and $L_2 = (SL_2, C_2)$, we have that $L_1 \sqsubseteq L_2$ iff $SL_1 \leq SL_2$ and $C_1 \subseteq C_2$, where C_1, C_2 are subsets of $\{A, I\}$ and SL_1, SL_2 are elements of $\{TS, S, C, P\}$.

A drawing of the subset partial order such that for any $L_1 = (SL_1, C_1)$ and $L_2 = (SL_2, C_2)$, we have that $L_1 \sqsubseteq L_2$ (i.e., there is an arrow from L_1 to L_2) iff $SL_1 \leq SL_2$ and $C_1 \subseteq C_2$, where C_1, C_2 are subsets of $\{A, I\}$ and SL_1, SL_2 are elements of $\{TS, S, C, P\}$.

2b [6 points] Use the BLP model above to answer these questions.

1. **[2 points]** Consider a user U with label $L_U = (C, \{A, I\})$ and an object O with label $L_O = (S, \{A, I\})$. Justify if user U can read object O and if user U can write object O .
2. **[1 points]** Provide an example of a label for a user U and a label for an object O such that U is allowed to read O .
3. **[1 points]** Provide an example of a label for a user U and a label for an object O such that U is not allowed to read O .
4. **[1 points]** Provide an example of a label for a user U and a label for an object O such that U is allowed to write O .
5. **[1 points]** Provide an example of a label for a user U and a label for an object O such that U is not allowed to write O .

Solution For the first point, we can see that $L_U \not\sqsubseteq L_O$ and $L_O \not\sqsubseteq L_U$, because S and C are not comparable, hence user U neither read nor write object O . For the other points, possible examples are: 2. $L_U = (S, \{I\})$ and $L_O = (P, \{I\})$; 3. $L_U = (S, \{I\})$ and $L_O = (P, \{A\})$; 4. $L_U = (P, \{I\})$ and $L_O = (S, \{I\})$; 5. $L_U = (S, \{I\})$ and $L_O = (P, \{A\})$.

- 3 Web Security [14 points]** Kungliga Operan (Royal Opera House of Sweden) has employed Alice with the task of securely setting up the ticket system for their customers. The system uses a database with one table with the schema below:

```
CREATE TABLE tickets (  
    customerName TEXT,  
    operaName TEXT,  
    -- Other columns not shown.  
);
```

To find all the operas that a customer has a ticket for, the website runs the following query (directly replacing `$customerName` with the user provided input):

```
SELECT operaName FROM tickets  
WHERE customerName = '$customerName';
```

Recall that the single quotes are used to delimit string literals in database queries. For example, `'$customerName'` evaluates to the value stored in variable `customerName`, say "Bob", and creates the string `'Bob'`. Moreover, `--` denotes a comment and `AND` and `OR` denote the usual logical operators.

- 3a [4 points]** Mallory, the attacker, wants to learn the name of all operas for which a customer has bought at least one ticket. Provide a malicious input for variable `$customerName` in the above query that would return the name of every opera with at least one ticket for it in the `tickets` table. Explain how the malicious input would achieve Mallory's goal.

Solution The malicious input `' OR 1 = 1 --` will result in the execution of the database query `SELECT operaName FROM tickets WHERE customerName = '' OR 1 = 1`, thus causing all database rows to be returned. This is due to the semantics of the logical `OR` operator which causes the `WHERE` part of the query to always evaluate to true.

- 3b [4 points]** We now consider a different situation. The website contains a page that takes as input an opera name, runs the query below (on the server-side), and displays "N tickets sold" to the user (where N is the number of rows returned by the query).

```
SELECT customerName FROM tickets  
WHERE operaName = '$operaName';
```

Provide an input to `$operaName` that Mallory could use to determine if Bob has a ticket for the opera "Aida". Explain how the above query allows Mallory to learn if Bob has a ticket for Aida.

Solution

The input `Aida' AND customerName = 'Bob' --` results in the execution of the database query `SELECT customerName FROM tickets WHERE operaName = 'Aida' AND customerName = 'Bob'`, thus returning one row if Bob has a ticket. If the website displays "1 tickets sold" then Bob has a ticket for Aida, otherwise he does not.

- 3c [6 points]** Alice releases a quick code fix for the previous attack. Now the page always displays "Ticket count not available" rather than "N tickets sold". However, the original query still runs before the this new message is displayed to the user.

Mallory realizes that when she provides "Carmen" as the `$operaName` in input, the website consistently takes less than 2 seconds to display the message "Ticket count not available".

Using this information, provide an input to `$operaName` that Mallory could use to determine if Bob has a ticket for "Carmen". Explain how the above query allows Mallory to learn if Bob has a ticket for Carmen.

For this part, you can assume that the database server supports the `SLEEP(X)` command that pauses the query evaluation for `X` seconds and then returns the number 1. Any additional assumptions should be stated explicitly.

Solution

The input `Carmen' AND customerName = 'Bob' AND SLEEP(10) = 1 --` results in the execution of the database query

```
SELECT operaName FROM tickets WHERE  
customerName = 'Carmen' AND customerName = 'Bob' AND SLEEP(10) = 1
```

If the website sleeps for 10 seconds and displays the message "Ticket count not available", then Bob has a ticket for Carmen, otherwise he does not. Note that this holds under the additional assumption that `SLEEP(10)=1` is not executed if the right-hand side of `AND` evaluates to false.

4 Hash Functions [12 points]

Let x be an input of n bits. Let $hash(x)$ be the following function.

```
def hash(x)
    H = SHA256(x)
    H = H ++ LSB128(x)
    return H
```

- 4a [12 points] Here, SHA256 is SHA-2 256 bits, LSB128 returns the last 128 bits of x , and ++ is bit concatenation. For each requirement of a cryptographically secure hash function, define the requirement, and either disprove the requirement or argue why the requirement is met.

Solution

- One way: given h it should be infeasible to find x such that $H(x) = h$. It is one way, even if weaker than SHA, since the first 256 bits of h are directly extracted from the output of SHA
- Weak collision resistance: given x it should be infeasible to find a different y such that $H(x) = H(y)$. It is weak collision resistant, since if one can find y , then $SHA256(x) = SHA256(y)$.
- Strong collision resistant: it should be infeasible to find any x and y such that $H(x) = H(y)$. It is strong collision resistant, since if one can find such pair $SHA256(x) = SHA256(y)$

- 5 Security Protocols [12 points] Alice wants to submit an assignment to a **Course Server (CS)**. Submissions may later be checked by an independent **Exam Office (EO)** in case of disputes.

Each party has a public/private key pair:

- Alice: (pk_A, sk_A)
- Course Server: (pk_{CS}, sk_{CS})
- Exam Office: (pk_{EO}, sk_{EO})

All public keys are known to all parties.

The protocol aims to ensure:

- Only Alice can submit an assignment.
- The Course Server cannot modify Alice's submission.
- The Exam Office can later resolve disputes about submissions.

Protocol)

1. Submission Alice prepares:

$$M = (\text{Alice}, \text{Assignment}, T)$$

She sends:

$$\text{Alice} \rightarrow \text{CS} : M, \text{Sig}_{sk_A}(M)$$

2. Acknowledgement Upon receiving the submission, CS verifies Alice's signature. To reduce storage costs, CS does *not* store the full message M . Instead, it computes a digest and returns:

$$\text{Ack} = (\text{Alice}, \text{CourseID}, H(M), T')$$

$$\text{CS} \rightarrow \text{Alice} : \text{Ack}, \text{Sig}_{sk_{CS}}(\text{Ack})$$

3. Audit In case of a dispute, Alice forwards the following to the Exam Office:

$$\text{Alice} \rightarrow \text{EO} : M, \text{Sig}_{sk_A}(M), \text{Ack}, \text{Sig}_{sk_{CS}}(\text{Ack})$$

5a [3 points] What attacks would be possible if Alice did *not* sign M ?

5b [3 points] Assume the Course Server later claims that Alice never submitted her assignment. Can the Exam Office resolve the dispute using the information from step 3? If not, how can you fix the protocol?

5c [3 points] Assume Alice later claims that she did not submitted her assignment. In this case, the Course Server sends the ack to the Exam Office:

$$\text{Ack} = (\text{Alice}, \text{CourseID}, H(M), T')$$

$$\text{CS} \rightarrow \text{EO} : \text{Ack}, \text{Sig}_{sk_{CS}}(\text{Ack})$$

Can the Exam Office resolve the dispute using the information from this step? If not, how can you fix the protocol?

5d [3 points] Suppose that there are multiple Course Servers $CS_{i \in [1 \dots n]}$ for n different exercises. Assume Alice later claims that she did not submitted her assignment for exercise i . How the protocol can be extended to allow CS_i to prove that Alice has submitted a specific assignment i ?

Solution (a) If Alice does not sign M , then CS has no cryptographic proof that M was authorised by Alice. Attacks include: Impersonation: anyone can submit a message claiming to be Alice; Tampering in transit: an attacker can modify Assignment

(b) Yes, EO can check that the signature of Alice matches M and that the signature CS matched Ack. Notice that since CS does not know the private key of Alice, it cannot compute Ack without the help of Alice.

(c) No, because EO also needs the original message M . CS must store M and send it with the Ack.

(d) The problem here is that when CS_i receives assignment i , it can use it to impersonate Alice forwarding the message to CS_j . To fix the problem, Alice must also send i (and sign) in the first message, later CS must provide back to EO Ack, i , and M .

6 Memory safety [14 points]

The following program is not memory safe. Here:

- `malloc` allocates heap memory, while `free` releases memory
- `memcpy(tmp, input, len)` copies `len` bytes starting from address `input` to address `tmp`
- `strcat(buffer, input);` iterates `buffer` until the `\0` terminator, then it copies `input` until the first `\0` terminator in `input`
- `input` and `len` are two inputs controlled by an attacker

```
1 void process(char *input, size_t len) {
2     char buffer[32];
3     char *tmp = malloc(32);
4     if (tmp == NULL)
5         return;
6     memcpy(tmp, input, len);    // Line A
7     free(tmp);                 // Line B
8     strcat(buffer, input);      // Line C
9     printf("%s\n", buffer);    // Line D
10    return;
11}
```

- 6a [2 points]** Identify the lines of the program that have a problem and explain why they are not secure.
- 6b [3 points]** Explain how an attacker can exploit the vulnerabilities in this function to gain control of the program's execution. Specify: which memory region is corrupted, what part of the state is overwritten, how execution is eventually redirected.
- 6c [3 points]** Explain how an attacker can use these vulnerabilities to inject and execute new code. In your answer include a description of the assumption you have made about the binary code and memory layout of the program.
- 6d [2 points]** Explain if (and why/why not) the attack in 6.c can be countered by randomizing the base address of the stack.
- 6e [2 points]** Explain if (and why/why not) the attack in 6.b can be countered by using guard pages.
- 6f [2 points]** If any of the two attacks cannot be countered by the proposed countermeasures, propose and motivate a different countermeasure.

Solution (a) Line A: tmp points to 32 bytes but len is attacker-controlled. If $\text{len} > 32$, this is a heap buffer overflow .

Line C Two issues: buffer is uninitialised, so strcat may read out of bounds while searching for the first NULL terminator; strcat performs no bounds checking: appending attacker-controlled input can cause a stack buffer overflow of buffer[32].

(b) The most direct path is via stack overflow at Line C: Attacker supplies input such that strcat(buffer, input) writes beyond the 32 byte buffer. The overflow can overwrite saved frame pointer and the saved return address in the current stack frame. The attacker overwrites the saved return address with an address of their choosing (e.g., an existing function). When process returns, the CPU jumps to that attacker-chosen address.

(c) Assuming the stack is executable, the attacker injects the binary in input (after the part that corrupt the return address) and use attack n (b) to point the return to input itself.

(d) For the attack in (c) yes, since the attacker will not know the address of buffer. However, since the payload is also pided in tmp, the attacker can point the return to tmp (assuming the attacker knows the address on the heap), since stack randomization does not affect the heap.

(e) No, since the overflow is on the stack. However, they may be used if placed between heap data structure to detect the overflow of tmp

(f) Simple solution would be stack canary. Better solution is to initialize buffer, to check that len is less than 32, and to limit strcat to only write up to 32 bytes.

7 Side channels [12 points] A system uses a shared, direct-mapped L1 data cache with:

- Total size: 1024 bytes
- Line size: 64 bytes
- Number of lines: 16
- Each cache line holds 16 int values (4 bytes each).
- The following global variables are defined:

```
int T[64];      // Public table: T[i] = i      mapped from line 0 to line 3
int secretA;    // Secret in range 0..63      mapped on line 15
int secretB;    // Secret in range 0..63      mapped on line 15
int dummy;      // Secret output              mapped on line 15
int x;          // Public input controlled by attacker mapped on line 15
```

The attacker knows that the victim runs the following function after each trigger:

```
1  if (x < 64) {
2      dummy = T[ secretA ];          // Line 1
3      dummy += T[ x xor secretB ];    // Line 2
4  }
```

The attacker can: (1) Flush the entire data cache before each run, (2) Choose the value of x for each run, (3) Observe which cache lines are accessed at least once during the run (access-driven attack), (4) Repeat the experiment many times with the same `secretA` and `secretB`.

- 7a [3 points]** Which lines of the program cause leakage of `secretA` or `secretB`? Which lines are safe? Explain briefly why each memory access does or does not reveal secret information.
- 7b [4 points]** Describe how an attacker can use chosen values of x and repeated measurements to extract: information about `secretA`, and information about `secretB`.
For each, specify: which cache lines become observable, what information is learned about the secret, why multiple runs with chosen x improve the attacker's knowledge.
- 7c [3 points]** Suggest a countermeasure that would prevent or significantly reduce the leakage. Explain why each prevents the attacker from learning information about `secretA` or `secretB`.
- 7d [2 points]** Explain if the program can leak more information in case the processor supports speculative execution (i.e., branch prediction).

(a) Line 2: `dummy = T[secretA]`; Leaks information about `secretA` because the accessed address depends on `secretA`,

Line 3: `dummy += T[x xor secretB]`; Leaks information about `secretB` because x is attacker-chosen and the index $x \oplus \text{secretB}$ determines which cache line of `T` is accessed

(b) **How the attacker extracts information about `secretA` and `secretB`.**

Let $L(i) = \lfloor i/16 \rfloor$ denote the cache line index of `T[i]`.

After one execution with input x , the attacker observes a set of accessed cache lines:

$$S_v = \{L(\text{secretA}), L(x \oplus \text{secretB}) \text{ for } x = v\}.$$

Learning information about `secretA`. The value $L(secretA)$ is the same across all executions, regardless of x . By running the victim with many different values of x and intersecting the observed sets, the attacker obtains:

$$L(secretA) \in \bigcap_v S_v.$$

where S_v are the lines accessed for the value $x = v$. With enough different inputs, the intersection contains only $L(secretA)$, revealing which block of 16 values `secretA` lies in.

Learning information about `secretB`. Once $L(secretA)$ is known, the attacker can determine $L(x \oplus secretB)$ for each run:

- If S_v contains two elements, the element different from $L(secretA)$ must be $L(x \oplus secretB)$.
- If S_v contains only one element, then $L(x \oplus secretB) = L(secretA)$.

Each execution yields a constraint of the form:

$$L(x \oplus secretB) = \ell_x,$$

where ℓ_x is observed. By trying all candidate values of `secretB` in the range 0..63 and keeping only those consistent with all observed constraints, the attacker can recover `secretB` completely.

(c) Possible effective countermeasures include Constant cache footprint: Always access all cache lines of `T` (e.g., touch `T[0]`, `T[16]`, `T[32]`, `T[48]`) before or instead of the secret-dependent accesses. This makes the set of accessed cache lines independent of secret values.

(d) If the processor supports speculative execution, the CPU may speculatively execute the body of the `if (x < 64)` statement even when the condition is false. During speculative execution, the loads from `T[secretA]` and `T[x xor secretB]` may still be performed and bring corresponding cache lines into the cache. The second load can cause a buffer overread, however this result is not transmitted via subsequent memory accesses.

8 Social engineering - I'll be (call-)back [12 points]

Eric gets the following mail:

Subject: Order confirmation
From: orders <no-reply@northernfitness.se>
To: eric.northman@gmail.com

Hej Eric!

Welcome to Northern Fitness! Here comes your order confirmation:

Order Number: #491625
Account Number: #366481
Items Ordered: Well-Fit Supreme Unlimited Membership
Start Date: Dec 15, 2025
End Date: Dec 15, 2027
Amount: SEK 9999,-
Payment Method: Klarna

Thank you for joining the Northern Fitness Family! Please note that we require indoor shoes in the facilities. That's all you need to bring, we'll provide you with a towel and a water bottle when you come to your first training.

In case you have questions, feel free to call our Well-Fit Supreme customer service at 070xxxxxxx [redacted for exam]. We can also help you get reimbursements of your friskvårdsbidrag (the 2025 deadline is December 16).

Happy lifting! Arnold Service heavyweight Northern Fitness

This is an automatically generated e-mail, you cannot reply to this.
We will never include links or attachments in our e-mails to you.
We take pride in our personal service.

Eric has not ordered this and calls the number. He gets put on hold to music, The Eye of the Tiger, but soon gets through.

Arnold: Welcome to Northern Fitness, this is Arnold, how can I help you?

Eric: Hej! I got a mail about a membership but I didn't order any.

Arnold: That's strange, let's see. Maybe someone of your friends and family signed you up! People get a bit pushy about fitness towards the New Year, I hope they don't also push a diet on you like my colleagues and their constant talk about carbs! What is your name and the last four digits of your person number? Just to confirm that it's really you.

(Eric gives him the digits. Arnold looks up Eric's phone number on hitta.se and gets his birthdate to complete the person number.)

Arnold: Yes, I see you have the Well-Fit Supreme membership for two years, our most luxurious package! And the payment has come through from your Klarna account already, well done!

Eric: No, no, I didn't order this, I don't want it, please cancel.

Arnold: Are you sure? It does come from your account. But you have the right to cancel online orders within 2 weeks, so we can make that happen. I'll e-mail you the cancelation form and you can fill it out when you are at your computer. Once we get that, we'll authorize the reimbursement, you'll get a BankID request to confirm.

Eric opens the mail and the attachment, fills out the form that pops up, clicks the button to confirm the cancelation. A QR code pops up and prompts him to open his BankID. He confirms the request Northern Fitness Reimbursement - SEK 9999.

8a [9 points] Identify 3 strategies (1 point each) used by the social engineer, for each identify the human trait they exploit (1 point each), and quote or mention the parts of the scenario you refer to (1 point each).

Solution Sample answers (don't need to be in these exact words):

1. pretext: trappings of the role (attribution, trust) by e.g. plausible story, no links or attachments, statement about links or attachment, marketing style in the e-mail, on-hold music
2. threat/fear (reduced cognitive ability), by e.g. saying the payment of SEK 9999 has already come through
3. building rapport/showing empathy/light banter (people agree to more from someone they like), talking about people pushing fitness or diets
4. pretext: put into victim role (reactance, altercasting) by e.g. creating and pretending to fix a problem.

8b [1 point] What did the attacker get?

Solution The attacker got Eric to open a file attached to an e-mail, confirm a money transfer with BankID, the full person number, and the phone number. Counted as correct if at least one of the listed gains is mentioned and no incorrect ones.

8c [2 points] Give two examples of what the attacker can do now.

Solution The attacker can have hidden malware, e.g. ransomware or attempts to get login credentials, in the attachment even if Eric does not authorize the bank transfer, get the money if he has, and mount identity-theft attacks in places that do not verify a person number with any other factor of authentication.